

Operación

Lo que hay que hacer a mano, y por qué.

Dos cosas están esperando: **crear dos secretos** y **medir un lote real** antes de encender los cron. Este documento cubre ambas.

1. Crear los dos secretos

Los dos van en **Sud-Austral/coipo_aireadme**, que es donde corren los workflows. Ninguno se necesita para el flujo normal de README: solo para el censo y el barrido.

Secreto	Para qué	Permiso mínimo
GH_FLEET_TOKEN	Censar y despertar repositorios	Lectura de la organización + escritura en Actions
GH_INDEX_TOKEN	Publicar el índice	Escritura sobre coipo_index y nada más

Son dos y no uno **a propósito**: el que publica el índice no necesita ver los 57 repositorios, y el que los lee no necesita poder escribir. Si alguna vez uno se filtra, el daño queda acotado.

GH_FLEET_TOKEN

GITHUB_TOKEN no sirve: no lee repositorios hermanos, y **30 de los 57 en alcance son privados**.

Opción A — token de grano fino (recomendado)

1. <https://github.com/settings/personal-access-tokens/new>
2. **Resource owner**: **Sud-Austral** (si aparece "requires approval", un propietario de la organización tiene que aprobarlo antes de que funcione)
3. **Repository access**: All repositories
4. **Permissions** → Repository permissions:

Permiso	Nivel	Por qué
Metadata	Read-only	obligatorio, se activa solo
Contents	Read-only	descargar el tarball para censar
Actions	Read and write	el barrido dispara readme.yml
Pull requests	Read-only	saber quién cerró un PR sin fusionar

5. **Expiration**: 90 días. Anótalo: cuando caduque, el censo y el barrido fallan con un mensaje explícito, no en silencio.

Opción B — token clásico

Más simple pero mucho más amplio: <https://github.com/settings/tokens/new> con los scopes **repo** y **workflow**. Concede escritura sobre todo lo que puedas tocar, así que prefiere la opción A si la organización la permite.

Guardarlo:

```
gh secret set GH_FLEET_TOKEN --repo Sud-Austral/coipo_aireadme  
# pega el token cuando lo pida; no queda en el historial del shell
```

GH_INDEX_TOKEN

1. <https://github.com/settings/personal-access-tokens/new>
2. **Resource owner:** **Sud-Austral**
3. **Repository access:** Only select repositories → **coipo_index**
4. **Permissions:** Contents **Read and write**, Metadata Read-only
5. Guardarlo:

```
gh secret set GH_INDEX_TOKEN --repo Sud-Austral/coipo_aireadme
```

Comprobar que quedaron

```
gh secret list --repo Sud-Austral/coipo_aireadme
```

Deben aparecer los dos. Para probarlos de verdad, lanza el censo acotado de la sección siguiente: si falta un secreto, el workflow lo dice y para, en vez de fallar a medias.

2. Medir un lote real

Los dos cron están comentados a propósito. Encenderlos sin medir es la forma más rápida de que alguien apague el sistema entero por saturación.

La pregunta que hay que responder no es "¿funciona?", sino "**¿cuánto cuesta y cuánto ruido genera?**".

2a. El barrido

Paso 1 — en seco. No dispara nada.

```
Actions → Barrido de la flota → Run workflow  
batch_size: 5  
dry_run: true
```

En el resumen de la ejecución verás a quién despertaría y por qué. Revisa que la lista tenga sentido antes de seguir.

Paso 2 — un lote de 3. Empieza pequeño.

```
batch_size: 3
dry_run: false
```

Paso 3 — recoger los números. Espera a que terminen las tres, y anota:

```
# Cuánto tardó cada una y cómo acabó
gh run list --repo Sud-Austral/<repo> --workflow=readme.yml --limit 1 \
  --json displayTitle,conclusion,startedAt,updatedAt

# Cuántos pull requests se abrieron en toda la flota
gh search prs --owner Sud-Austral --state open \
  --head ai-readme/update-readme --json repository,url
```

Y del resumen de cada ejecución (pestaña Actions → la corrida → Summary), que trae el informe completo:

Qué medir	Dónde	Qué esperar
Duración por repositorio	<code>gh run list</code>	2–5 min
Cuántos generaron de verdad	resumen: "README creado" o "Bloques actualizados"	—
Cuántos no tocaron nada	resumen: "No se modificó nada" o "SIN CAMBIOS"	los de README humano
Pull requests abiertos	<code>gh search prs</code>	≤ los que generaron
Fallos por límite del modelo	log: "Intento N/4 falló"	0 es lo esperable

Paso 4 — decidir. Con esos números:

- Si de 3 repositorios **2 o más no tocaron nada**, el cron es barato: la mayoría de las corridas cortan antes de llamar al modelo.
- Si aparecen **reintentos por 429**, baja `max-parallel` de 3 a 2 en `fleet-sweep.yml` antes de subir el lote.
- Si los pull requests que se abrieron **no valen la pena**, el problema no es el cron: es que esos repositorios no deberían estar en el barrido. Ciérralos sin fusionar y el sistema deja de proponérselos.

Repite con `batch_size: 10`. Si se sostiene, **entonces** descomenta el cron en `.github/workflows/fleet-sweep.yml`:

```
schedule:
  - cron: "0 6 * * 1" # lunes 06:00 UTC
```

Cuenta rápida para dimensionarlo. 57 repositorios, ~4 min cada uno, con 3 en paralelo son unos **75 minutos de Actions** por barrido completo. En repositorios públicos eso no consume cuota; en los 30 privados sí. Un cron semanal que solo despierta a 10 es del orden de 13 minutos por semana.

2b. El censo

Paso 1 — 5 repositorios, sin publicar.

```
Actions → Censo de la flota → Run workflow
  limite:    5
  publicar:  false
```

Deja el resultado como artefacto descargable. Mide la duración.

Paso 2 — **extrapolar**. Multiplica por 11 para los 57.

De referencia, **medido en local el 6 de septiembre de 2026**:

6 repositorios	~50 segundos
57 repositorios	~25 minutos
Fichas producidas	49
No censados	8, todos con su motivo en el inventario

Los 8 fueron 6 que superan los 293 MB —**coipo_canciones** pesa **2,5 GB**, casi todo datos versionados— y 2 que fallaron al descargar. El límite está en **MAX_TAMANO_KB** de **fleet_scan.py**; súbelo si quieres alcanzarlos, a cambio de disco y tiempo en el runner.

Paso 3 — **publicar**.

```
limite:    0
publicar:  true
```

Escribe las fichas en **coipo_index** y el catálogo público en **fleet/**. Antes de publicar comprueba que ningún dato de repositorio privado se coló y **aborta si encuentra algo**.

Si se sostiene, descomenta el cron en **fleet-census.yml**. Semanal es de sobra: la deuda documental no cambia a diario.

3. Cómo probar sin arriesgar nada

Antes de disparar nada sobre un repositorio, puedes ver exactamente qué haría:

```
python preview.py "D:/GitHub/COIPO_ENTREGA_PLANTA"
```

No llama al modelo, no escribe nada y responde: qué evidencia ve y con qué procedencia, qué decidiría el contrato con el humano, si la evidencia cambió desde la última generación, qué archivos propondría borrar y qué artefactos propondría.

4. Qué hacer cuando algo se rompe

Síntoma	Causa probable	Qué mirar
El workflow para diciendo que falta un secreto	no está creado o caducó	<code>gh secret list --repo Sud-Austral/coipo_aireadme</code>
Un repositorio en alcance no genera nada	contrato o huella	<code>python preview.py <ruta></code> lo dice en una línea
Todos dejan de generar de golpe	la huella o la guarda	skipped es gris, no rojo: no salta ninguna alarma. <code>python auditar_workflows.py</code>
Aparecen tecnologías falsas	regresión del motor	<code>python -m pytest tests/</code> — el fixture negativo es este mismo repositorio y exige cero
El censo aborta por fuga	un dato privado en la vista pública	el mensaje nombra el repositorio; revisa <code>fleet_card.verificar_sin_fuga</code>

El modo de fallo que más preocupa es el tercero. Un corte por huella mal calculado apaga la documentación de toda la flota **en silencio**, porque `sys.exit(0)` se ve idéntico a un éxito en la interfaz de Actions. Si un lunes nadie generó nada, empieza por ahí.

5. Salir del sistema

Para que un repositorio deje de recibir el generador, en orden de menos a más definitivo:

1. `.aireadme.yml` con `enabled: false`. Para la generación, pero el workflow sigue invocándose: no es una salida de confianza.
2. `<!-- ai-readme:lock -->` en el README. Lo respeta antes de llamar al modelo.
3. **Borrar** `.github/workflows/readme.yml`. La salida de verdad.

Escribir el README a mano ya basta para que no lo toque: el contrato solo escribe donde no hay documentación o donde alguien puso marcadores `AI:BEGIN` a propósito.